Campus Event Management & Feedback Analyzer

A PROJECT REPORT

Submitted by

ROLL NUMBER	REGISTRATION NO.	STUDENT CODE	STUDENT NAME
26102410042	**26102000164**	**BWU/MCA/26/047**	**Puskar Mondal**

in partial fulfillment for the award of the degree

of

MASTER OF COMPUTER APPLICATIONS

IN

DEPARTMENT OF COMPUTATIONAL SCIENCES

BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

TABLE OF CONTENTS

Chapter	Title
1	**INTRODUCTION**
	1.1 INTRODUCTION
	1.2 Background of the Project
	1.3 Problem Statement
	1.4 Proposed Solution
	1.5 Scope of the Project
	1.6 Importance of the Project
	1.7 Organization of the Report
2	**OBJECTIVE**
	2.1 Main Objective
	2.2 Event Management
	2.3 Student Registration Management
	2.4 Student Search and Registration Cancellation
	2.5 Feedback Collection
	2.6 Feedback Analysis using Pandas
	2.7 Graphical Representation
3	**PLANNING**
	3.1 Project Planning
	3.2 Development Approach
	3.3 Technology Selection

3.4 Project Modules

3.5 Data Storage Planning

3.6 Development Environment & Execution Planning

3.7 Project Limitations and Planning Consideration

4 REQUIREMENT ANALYSIS

4.1 Introduction to Requirement Analysis

4.2 Functional Requirements

4.3 Non-Functional Requirements

4.4 Hardware Requirements

4.5 Software Requirements

4.6 Data Requirements

4.7 System Constraints and Limitations

Chapter **Title**

5. **SYSTEM FLOW**

5.1 Introduction

5.2 Overall System Flow

5.3 Event Creation Flow

5.4 Student Registration Flow
5.5 Student Search and Registration Cancellation Flow
5.6 Feedback Collection Flow
5.7 Feedback Analysis Flow
5.8 Data Flow and Storage

6. **SYSTEM DESIGN**

6.1 Introduction

6.2 System Archietecture
6.3 System Archietecture Diagram
6.4 Module Design
6.5 Class Design
6.6 Class Diagram
6.7 Data Design
6.8 User Interface Design
6.9 Technology Design

7. **EXPERIMENTAL RESULT**

7.1 Introduction
7.2 Experimental Environment
7.3 Homepage Result
7.4 Event Creation Result
7.5 Student Registration Result
7.6 Student Search Result
7.7 Registration Cancellation Result
7.8 Feedback Collection Result
7.9 Feedback Analysis Result
7.10 Graphical Representation Result
7.11 Testing Result
7.12 Ovarall Experimental Outcome

8. CONCLUSION

CHAPTER 1: INTRODUCTION

1.1 Introduction

In today's digital world, technology plays an important role in improving the efficiency and management of various activities in educational institutions. Universities and colleges regularly organize different types of events such as cultural festivals, technical seminars, workshops, sports competitions, annual functions, and educational programmes. These events provide students with opportunities to develop their knowledge, skills, creativity, leadership qualities, and teamwork abilities.

Managing these events manually can become a challenging task, especially when a large number of students participate. Maintaining event details, recording student registrations, checking available seats, managing participant information, and collecting feedback require proper organization. When these activities are handled using traditional paper-based methods or separate records, there may be difficulties in maintaining accurate information, finding student details, identifying duplicate registrations, and calculating event participation.

To address these challenges, the CampusConnect – Campus Event Management & Feedback Analyzer project has been developed as a web-based application using Python and the Flask framework. The main purpose of this system is to provide a simple and organized platform for managing campus events, student registrations, and event feedback through a web browser.

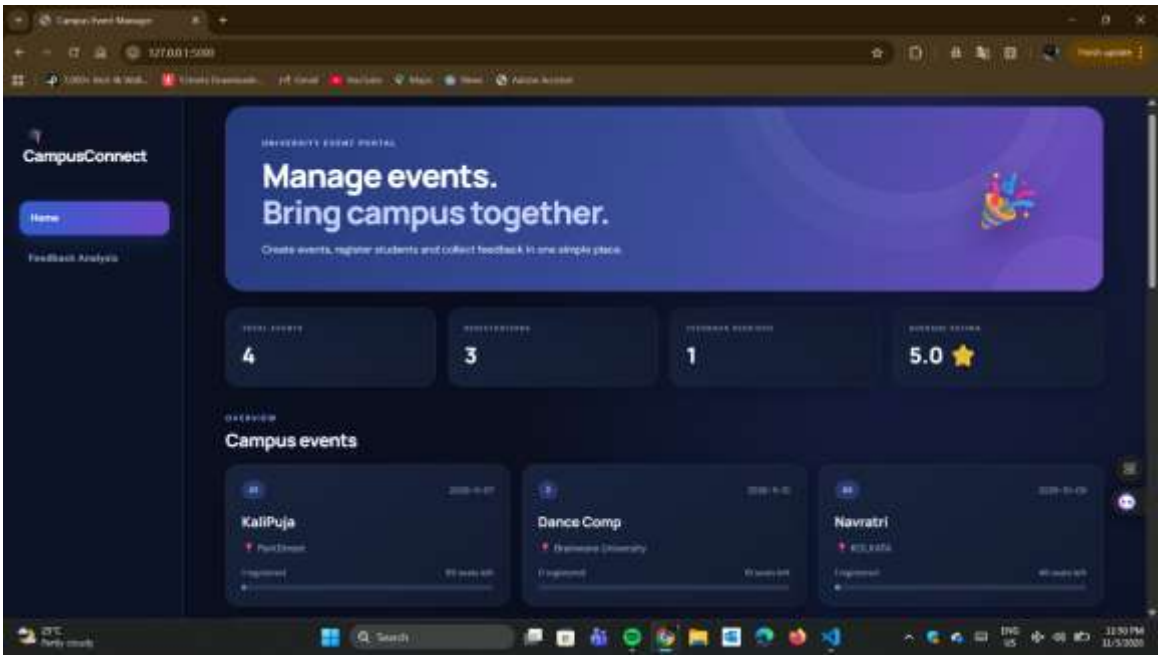
The application allows users to create and manage event information by entering details such as Event ID, Event Name, Date, Venue, and Maximum Participants. Students can register for available events by providing their basic information, including Student ID, Name, Email, and Department. The system checks whether an event exists, whether the student has already registered for the same event, and whether the event has reached its maximum participant capacity.

Apart from event management and student registration, the application also provides facilities for searching student information and cancelling existing registrations. The search functionality helps users find student details using a Student ID, while the cancellation functionality allows an existing registration to be removed by providing the relevant student and event information.

Another important feature of CampusConnect is its feedback management system. Students who have registered for an event can submit a rating to express their experience. The application prevents feedback from students who are not registered for the selected event and also avoids multiple feedback submissions by the same student for the same event.

The system uses CSV files to store event details, student information, registration records, and feedback data. Pandas is used to process feedback information and calculate useful statistics, such as average ratings and rating counts. Matplotlib is integrated to generate a graphical representation of event-wise participation.

Overall, CampusConnect combines event management, student registration, feedback collection, data analysis, and graphical visualization in one web-based platform. It demonstrates how Python programming and web development technologies can be used to develop a practical solution for managing common campus activities.



1.2 Background of the Project

Educational institutions conduct numerous academic and extracurricular activities throughout the year. These activities are important for student development and help create an active and engaging campus environment.

However, organizing an event involves several administrative tasks. The organizers need to maintain event information, record student details, monitor the number of participants, and manage registration-related activities. After an event is completed, feedback is also useful for understanding student satisfaction and identifying areas that may require improvement.

In a traditional event management process, these activities may be performed using notebooks, spreadsheets, or separate digital records. As the number of events and participants increases, maintaining these records can become time-consuming. It may also become difficult to identify duplicate registrations, check the availability of seats, and prepare participation reports.

The development of web-based applications provides an opportunity to simplify these activities. A web application can organize different operations through a common interface and allow users to perform tasks using a browser.

CampusConnect has been developed with this idea in mind. It provides a centralized interface for basic campus event operations and maintains the related information using CSV-based storage. The application also includes feedback analysis facilities to make the collected information more useful.

The project is designed as an academic application that demonstrates the practical use of Python, Flask, file handling, object-oriented programming, Pandas, and Matplotlib.

1.3 Problem Statement

Managing campus events involves multiple activities, and performing these activities manually can create several difficulties. When event information and student registration records are maintained separately, organizers may face problems in finding accurate information and updating records.

Some common problems associated with manual event management are:

1. Difficulty in Managing Event Information: Maintaining event names, dates, venues, and participant limits in separate records can make information management difficult.

2. Duplicate Student Registration: Without proper validation, the same student may register for the same event more than once.

3. Event Capacity Management: Organizers need to monitor the number of registered participants to ensure that the maximum participant limit is not exceeded.

4. Difficulty in Searching Student Details: Finding a particular student's registration information from large records may require additional time and effort.

5. Registration Cancellation: When a student cancels participation, the registration record needs to be updated correctly.

6. Feedback Management: Collecting and organizing feedback from participants can become difficult when feedback information is maintained manually.

7. Difficulty in Analysing Feedback: Calculating average ratings and understanding feedback patterns manually may be time-consuming.

8. Lack of Graphical Representation: Numerical participation records may not provide an immediate understanding of which events have higher or lower participation.

Therefore, there is a need for a simple web-based system that can organize event information, manage student registrations, maintain feedback records, and provide basic analytical results.

1.4 Proposed Solution

To overcome the above-mentioned problems, the CampusConnect web application has been developed using Python and Flask. The proposed system provides different features through a common web interface.

The application includes the following major functionalities:

- Event Creation: Users can create new campus events by entering the required event details. The system checks for duplicate Event IDs before saving a new event.
- Student Registration: Students can register for available events by submitting their personal and academic information.

- Registration Validation: The application checks event availability, duplicate registrations, and maximum participant capacity before accepting a registration.
- Student Search: Users can search for student information using a Student ID and view the associated registration details.
- Registration Cancellation: Existing registrations can be cancelled by providing the Student ID and Event ID.
- Feedback Submission: Registered students can submit ratings for events. The system validates registration and prevents duplicate feedback submissions.
- Feedback Analysis: Pandas is used to calculate the average rating and count the feedback ratings.
- Participation Visualization: Matplotlib generates a bar chart representing the number of registrations for different events.

The application uses Flask routes to process user requests and HTML templates to display information. CSS is used to improve the appearance and usability of the web interface. CSV files are used for storing information locally, making the project simple to operate without requiring a separate database server.

1.5 Scope of the Project

The scope of the CampusConnect project is focused on the basic management of campus events and the analysis of student feedback. The application is developed primarily for academic purposes and demonstrates how a web-based system can simplify event-related administrative activities.

The major areas covered by the project are:

1. Event Information Management

The application provides a facility to create campus events and maintain their basic details, including Event ID, event name, date, venue, and maximum participant capacity.

2. Student Information Management

The system stores student information such as Student ID, name, email address, and department. This information is used during event registration and student searches.

3. Registration Management

Students can register for available events. The application applies validation rules to prevent duplicate registrations and registrations beyond the maximum event capacity.

4. Registration Search and Cancellation

The application allows users to search for student information and view associated event registrations. It also provides a facility to cancel an existing registration.

5. Feedback Collection

Students who have registered for an event can submit their ratings. The system maintains feedback records and checks whether a student has already submitted feedback for the same event.

6. Feedback Analysis

The application uses Pandas to process feedback records and calculate average ratings and rating counts.

7. Graphical Representation

Matplotlib is used to create an event-wise participation chart. This graphical representation helps users understand and compare the registration counts of different events.

The current version uses CSV files for local data storage and is intended for a small academic demonstration. Advanced features such as user authentication, role-based access, database integration, and online deployment are not included in the current implementation.

1.6 Importance of the Project

CampusConnect is useful because it demonstrates how different event-related activities can be organized within a single web application. Instead of maintaining separate records for events, students, registrations, and feedback, the application provides a structured approach to managing these details.

The project is also important from an educational perspective because it combines several programming concepts and technologies in one practical application.

It demonstrates the use of Python classes for representing event-related entities, Flask for developing web routes, CSV file handling for data storage, Pandas for data analysis, and Matplotlib for graphical visualization.

The feedback analysis feature provides a basic way to understand student responses, while the participation chart makes event registration information easier to interpret. These features demonstrate how stored data can be converted into useful information for decision-making.

1.7 Organization of the Report

This project report is divided into nine chapters. Each chapter explains a different aspect of the CampusConnect application.

- Chapter 1 – Introduction: Provides an overview of the project, its background, problem statement, proposed solution, scope, and importance.
- Chapter 2 – Objectives: Explains the main objectives and functional goals of the application.
- Chapter 3 – Planning: Describes the project planning process, development approach, technology selection, and project modules.
- Chapter 4 – Requirement Analysis: Discusses the functional requirements, non-functional requirements, hardware requirements, software requirements, and data requirements.

- Chapter 5 – System Flow: Explains the overall workflow of the application and the processes involved in event creation, student registration, cancellation, and feedback submission.
- Chapter 6 – Proposed Design: Describes the system architecture, entity design, CSV file structure, Flask routes, user interface, and analytical components.
- Chapter 7 – Experimental Result: Presents the implemented features and expected application behaviour for different operations.
- Chapter 8 – Future Scope: Discusses possible improvements and additional features that may be introduced in future versions.
- Chapter 9 – Conclusion: Summarizes the project, the technologies used, the learning outcomes, and the overall result.

CHAPTER 2: OBJECTIVES

2.1 Main Objective

The main objective of the CampusConnect – Campus Event Management & Feedback Analyzer project is to develop a simple, user-friendly, and efficient web-based application for managing campus events, student registrations, and event feedback.

The application is developed using Python and the Flask framework to provide a centralized platform where users can perform different event management operations through a web browser. It aims to reduce the difficulties associated with maintaining event information and student registration records manually.

The system focuses on organizing event details, maintaining student information, managing registrations, collecting feedback, and analysing the collected data. It also provides validation mechanisms to maintain the correctness of important operations, such as preventing duplicate event creation, duplicate student registration, and registration beyond the maximum participant capacity.

Another important objective is to demonstrate the practical application of Python programming concepts and web development technologies. By integrating Flask, CSV file handling, Pandas, and Matplotlib, the project provides an example of how a small web application can be developed to solve a real-world administrative problem in an educational institution.

2.2 Event Management

One of the major objectives of the project is to provide an organized facility for creating and maintaining campus event information.

In educational institutions, different events are organized throughout the academic year. Maintaining accurate information about these events is necessary for proper planning and registration management.

The application allows users to create new events by entering the following information:

- Event ID: A unique identification number assigned to each event.
- Event Name: The name or title of the event.
- Event Date: The date on which the event will take place.
- Venue: The location where the event will be organized.
- Maximum Participants: The maximum number of students allowed to register for the event.

The system checks whether the entered Event ID already exists in the event records. If the same Event ID is entered again, the application rejects the operation and displays an appropriate error message.

This validation helps maintain unique event identification and avoids confusion between different events.

The event management functionality also helps maintain organized records that can be used during student registration and participation analysis.

2.3 Student Registration Management

Another important objective of CampusConnect is to provide a simple and structured student registration facility.

Students can register for available campus events by providing their basic information. The application collects details such as Student ID, student name, email address, department, and the Event ID for which registration is required.

The system validates the registration information before accepting a new registration. It checks whether the selected event exists in the event records and whether the student has already registered for the same event.

The application also checks the maximum participant capacity of the selected event. If the event has already reached its registration limit, the system does not accept additional registrations.

The major objectives of this module are:

1. To provide an easy registration process for students.
2. To maintain student information in an organized manner.
3. To prevent duplicate registration for the same event.
4. To prevent registration for invalid or unavailable Event IDs.
5. To ensure that the maximum participant capacity is not exceeded.
6. To maintain registration records for future searching and analysis.

Through this module, the application helps organize student participation and maintain accurate registration information.

2.4 Student Search and Registration Cancellation

The project aims to provide facilities for searching student information and cancelling existing event registrations.

When an institution organizes multiple events, it may become difficult to find the registration details of a particular student from a large collection of records. The student search functionality helps simplify this process.

Users can enter a Student ID to search for a particular student. If the student exists in the stored records, the application displays the available student information and the event IDs associated with that student.

The search functionality helps users retrieve information without manually checking the complete registration records.

The application also provides a registration cancellation facility. If a student decides not to participate in a particular event, the corresponding registration can be cancelled by providing the Student ID and Event ID.

The cancellation operation removes the matching registration record from the registration data.

The objectives of this module are:

- To make student information retrieval easier.
- To identify the events in which a student is registered.
- To reduce the time required for searching registration records.
- To provide a simple way to cancel an existing registration.
- To maintain updated registration information after cancellation.

2.5 Feedback Collection

Collecting feedback from students is an important part of event management because it helps organizers understand the participants' experiences and satisfaction levels.

One of the objectives of CampusConnect is to provide a digital feedback submission facility for registered students.

After participating in an event, students can submit a numeric rating through the feedback section of the application. The submitted rating is stored along with the Student ID and Event ID in the feedback CSV file.

The application performs validation before accepting feedback. It checks whether the student is registered for the selected event. If the student does not have a valid registration, the application does not accept the feedback.

The system also checks whether the student has already submitted feedback for the same event. This prevents duplicate feedback entries and helps maintain more consistent feedback records.

The main objectives of the feedback module are:

1. To provide a simple digital method of collecting event ratings.
2. To collect feedback from registered participants.
3. To prevent feedback submission by non-registered students.

4. To prevent duplicate feedback from the same student for the same event.
5. To maintain feedback information for further analysis.
6. To support the evaluation of event performance through collected ratings.

This module helps make the event evaluation process more organized and reduces the need for maintaining feedback records manually.

2.6 Feedback Analysis Using Pandas

Another major objective of the project is to analyse the feedback information collected from students and convert the stored ratings into useful statistical information.

The application uses the Pandas library to read and process feedback records stored in the feedback CSV file. The feedback data is converted into a Pandas DataFrame so that different calculations can be performed easily.

The system calculates the average rating of the submitted feedback and counts the number of feedback entries. It also calculates the frequency of different rating values.

The average rating provides a basic indication of how students evaluated an event. The rating count helps understand how many students submitted each particular rating.

The main objectives of this module are:

- To read feedback information from the CSV file.
- To organize the feedback records into a DataFrame.
- To calculate the average event rating.
- To calculate the total number of feedback submissions.
- To count the frequency of different rating values.
- To present the analysis results in an understandable format.

The use of Pandas makes the feedback analysis process easier and demonstrates the practical application of data analysis techniques in a Python-based web application.

2.7 Graphical Representation Using Matplotlib

The final major objective of the project is to represent event participation information graphically using the Matplotlib library.

Numerical information stored in registration records can be difficult to compare when multiple events are available. A graphical representation helps users understand participation differences more easily.

CampusConnect uses Matplotlib to generate an event-wise participation bar chart. The chart represents the number of students registered for each event and provides a visual comparison of event participation.

The application counts the registrations associated with each event and uses these values to generate the chart. The generated chart is saved in the static directory so that it can be displayed through the web interface.

The major objectives of this module are:

1. To calculate the number of registrations for each event.
2. To generate a bar chart using Matplotlib.
3. To display event participation information visually.
4. To make comparison between different events easier.
5. To demonstrate the integration of graphical visualization in a Flask application.

This graphical feature improves the presentation of registration data and helps users understand event participation at a glance.

CHAPTER 3: PLANNING

3.1 Project Planning

Project planning is one of the most important stages in software development. It involves identifying the requirements, defining the objectives, selecting suitable technologies, organizing the development activities, and deciding how the final application will work.

The CampusConnect – Campus Event Management & Feedback Analyzer project was planned to develop a simple and user-friendly web-based system for managing campus events and student-related activities. The main purpose of the planning stage was to divide the complete project into smaller modules so that each functionality could be developed and implemented systematically.

The application was designed using Python and the Flask framework. The project also uses HTML for creating the structure of web pages, CSS for designing the user interface, CSV files for storing information, Pandas for analysing feedback data, and Matplotlib for graphical representation.

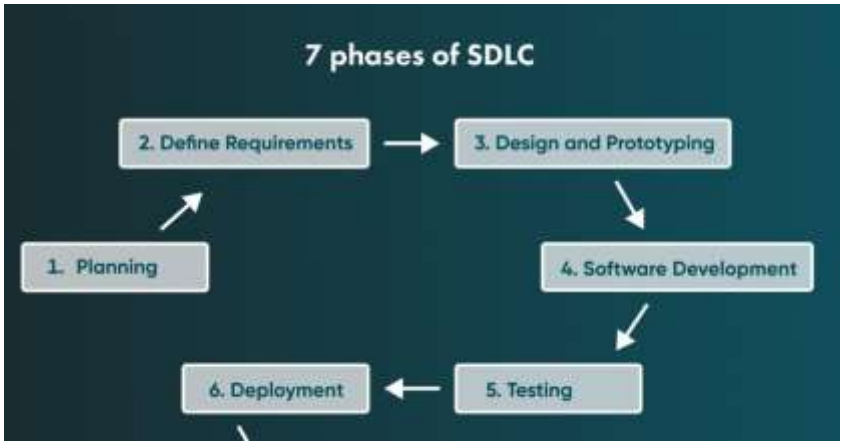
During the planning stage, the major functionalities of the application were identified. These include event creation, student registration, student search, registration cancellation, feedback submission, feedback analysis, and event-wise participation visualization.

The project was divided into different development stages:

1. Identification of the problem and project requirements.
2. Selection of suitable programming languages and technologies.
3. Planning the structure of the application.
4. Designing the required data storage files.
5. Developing the Flask application and its routes.
6. Creating the HTML templates and CSS interface.

7. Implementing event and student registration functionalities.
8. Integrating feedback collection and analysis.
9. Generating graphical reports using Matplotlib.
10. Reviewing the implemented features and preparing the final project report.

This systematic planning helped organize the development process and ensured that the major requirements of the project were addressed.



3.2 Development Approach

The CampusConnect application was developed using a modular and step-by-step development approach. Instead of implementing all functionalities at once, the application was divided into smaller components. Each component was developed according to its specific responsibility and then integrated into the main Flask application.

The development process started with the basic structure of the application. The required project folders and files were organized, and the Flask application was configured to handle browser requests.

After preparing the basic structure, the event management module was implemented. This module allows users to create events and store event information. Validation was also included to prevent duplicate Event IDs.

The next stage focused on student registration. The application was designed to collect student details and connect each registration with a valid event. Additional validation was implemented to prevent duplicate registrations and ensure that event capacity is not exceeded.

After completing the registration functionality, the student search and cancellation modules were developed. These features allow users to retrieve student registration information and remove existing registrations when required.

The feedback module was then implemented to collect event ratings from registered students. Validation was included to ensure that only registered students can submit feedback and that duplicate feedback is not accepted for the same student-event combination.

Finally, Pandas and Matplotlib were integrated into the application. Pandas is used to calculate feedback statistics, while Matplotlib generates the event-wise participation chart.

The development approach can therefore be divided into the following major stages:

- Requirement Identification: Understanding the problems associated with manual campus event management.
- System Design: Planning the application structure, modules, data files, and user interface.
- Backend Development: Implementing application logic and Flask routes using Python.
- Frontend Development: Creating web pages using HTML templates and CSS.
- Data Management: Implementing CSV-based storage for event, student, registration, and feedback records.
- Analysis and Visualization: Integrating Pandas and Matplotlib for feedback analysis and graphical representation.
- Final Review: Reviewing the implemented functionalities and preparing the application for demonstration.

This approach made the development process more organized and easier to manage.

3.3 Technology Selection

Selecting appropriate technologies is an important part of project planning. The technologies used in CampusConnect were selected according to the requirements of the application, ease of implementation, availability of libraries, and suitability for academic development.

The project uses Python as the main programming language and Flask as the web framework. Other technologies and libraries are used for designing the interface, storing information, analysing feedback, and generating charts.

3.3.1 Python

Python is the primary programming language used to develop the CampusConnect application. It provides a simple syntax and a wide range of libraries that make application development easier.

Python is used for implementing the main application logic, handling form submissions, performing validation, reading and writing CSV files, and processing student and event information.

The application also uses Python classes to represent important entities such as Event, Student, and Registration. This helps organize related information into structured objects.

3.3.2 Flask

Flask is a lightweight web framework used to develop the backend of the CampusConnect application.

It provides the functionality required to handle browser requests, process submitted forms, execute application logic, and render HTML templates.

Different Flask routes are used to perform operations such as displaying the dashboard, creating events, registering students, searching student information, cancelling registrations, submitting feedback, and displaying analysis results.

Flask was selected because it is lightweight, flexible, and suitable for developing small and medium-sized Python web applications.

3.3.3 HTML

HTML stands for HyperText Markup Language. It is used to define the structure and content of the web pages.

In CampusConnect, HTML templates are used to create the dashboard, event management forms, student registration forms, feedback sections, and analysis pages.

The templates are stored inside the templates directory and are rendered by Flask when users access the corresponding routes.

3.3.4 CSS

CSS stands for Cascading Style Sheets. It is used to control the appearance and presentation of the web application.

The project uses CSS to design the interface, including page layouts, colours, backgrounds, buttons, forms, spacing, and visual effects.

The stylesheet is stored in the static directory. It helps provide a consistent and professional appearance across the different sections of the application.

3.3.5 CSV File Handling

CSV stands for Comma-Separated Values. It is a simple file format used to store tabular information.

CampusConnect uses CSV files for maintaining event details, student information, registration records, and feedback data.

The application uses four main data files:

- events.csv
- students.csv
- registrations.csv
- feedback.csv

CSV storage was selected because it is simple to understand, easy to maintain, and suitable for a small academic project.

3.3.6 Pandas

Pandas is a Python library used for data manipulation and analysis.

In CampusConnect, Pandas is used to read and process feedback records. The feedback information is organized into a DataFrame so that calculations can be performed efficiently.

The application uses Pandas to calculate average ratings and count the frequency of different rating values.

3.3.7 Matplotlib

Matplotlib is a Python library used for creating graphical visualizations.

The project uses Matplotlib to generate a bar chart representing event-wise student participation. The chart displays the number of registrations for different events and makes it easier to compare participation levels.

The generated chart is saved inside the static directory so that it can be displayed through the web interface.

3.4 Project Modules

For better organization and development, the CampusConnect application is divided into several functional modules. Each module performs a specific operation, and all modules work together to provide the complete functionality of the application.

3.4.1 Event Management Module

The Event Management Module is responsible for creating and maintaining campus event information.

Users can enter the Event ID, Event Name, Date, Venue, and Maximum Participants through the event creation form.

Before storing a new event, the application checks whether the entered Event ID already exists. If a duplicate Event ID is found, the system rejects the operation and displays an error message.

This module helps maintain organized event records and provides the necessary event information for student registration.

3.4.2 Student Registration Module

The Student Registration Module allows students to register for available campus events.

The application collects Student ID, Name, Email, Department, and Event ID through a registration form.

Before accepting the registration, the system checks whether the event exists, whether the student has already registered for that event, and whether the event has available participant capacity.

If all validation conditions are satisfied, the student and registration information are stored in the appropriate CSV files.

3.4.3 Student Search Module

The Student Search Module provides a facility to search for student information using a Student ID.

When a user enters a valid Student ID, the application searches the stored student records and retrieves the associated registration information.

This module makes it easier to find student details without manually checking the complete registration file.

3.4.4 Registration Cancellation Module

The Registration Cancellation Module allows users to remove an existing event registration.

The user provides the Student ID and Event ID associated with the registration. The application searches for the matching registration record and removes it from the registration data.

This functionality helps maintain updated participation records when students decide not to attend an event.

3.4.5 Feedback Management Module

The Feedback Management Module allows registered students to submit ratings for campus events.

The application checks whether the student is registered for the selected event before accepting the feedback. It also checks whether feedback has already been submitted by the same student for that event.

The accepted feedback is stored in feedback.csv for further processing and analysis.

3.4.6 Feedback Analysis Module

The Feedback Analysis Module processes the collected feedback information using Pandas.

It calculates the average rating and the number of feedback submissions. It also provides rating-frequency information to help understand the distribution of student ratings.

This module converts the stored feedback records into useful statistical information.

3.4.7 Graphical Visualization Module

The Graphical Visualization Module uses Matplotlib to generate an event-wise participation chart.

The application counts the number of registrations associated with each event and represents the results through a bar chart.

This module makes it easier to compare the participation levels of different campus events.

3.5 Data Storage Planning

Data storage planning is an important part of the CampusConnect development process because the application needs to maintain different types of information.

The project uses CSV files to store data locally. A separate CSV file is maintained for each major category of information. This separation makes the records easier to organize, retrieve, and update.

The application automatically creates the required data directory and initializes the CSV files with their respective column headings when they do not already exist.

The following files are used:

1. events.csv

This file stores the details of all campus events.

Field Name	Description
event_id	Unique event identification
name	Name of the event
date	Date of the event
venue	Event location
max_participants	Maximum registration capacity

2. students.csv

This file stores the personal and academic information of students.

Field Name	Description
student_id	Unique student identification
name	Student name
email	Student email address
department	Student department

3. registrations.csv

This file stores the relationship between students and events.

Field Name	Description
student_id	ID of the registered student

event_id	ID of the selected event

4. feedback.csv

This file stores the ratings submitted by registered students.

Field Name	Description
student_id	ID of the student submitting feedback
event_id	ID of the event
rating	Numeric feedback rating

The use of separate CSV files helps maintain a clear structure for different types of records and supports the application's event management and feedback analysis operations.

3.6 Development Environment and Execution Planning

The CampusConnect application was planned to run in a local development environment using Python and a web browser.

The project source code is organized into different files and folders. The main application logic is maintained in app.py, while HTML templates and CSS files are stored separately.

The major project structure is as follows:

```
CampusConnect/
│
├── app.py
├── requirements.txt
├── README.txt
│
├── data/
│   ├── events.csv
│   ├── students.csv
│   ├── registrations.csv
│   └── feedback.csv
│
├── templates/
│   ├── index.html
│   └── analysis.html
│
└── static/
    ├── style.css
    └── participation.png
```

The requirements.txt file contains the Python packages needed to run the application. These dependencies can be installed using the Python package installer.

The general execution procedure is:

1. Open the project folder in a terminal or command prompt.
2. Install the required dependencies using the requirements file.

3. Start the Flask application by running app.py.
4. Open a web browser.
5. Visit the local Flask address displayed by the application.
6. Use the available web interface to access the different functionalities.

This execution arrangement makes the application suitable for local demonstration and academic project evaluation.

3.7 Project Limitations and Planning Considerations

During the planning and development of CampusConnect, the project was designed with a limited academic scope. The main focus was to implement the basic requirements of campus event management and feedback analysis using Python-based technologies.

The current application uses CSV files instead of a database management system. This approach is simple and suitable for a small demonstration, but it may not be appropriate for handling a large number of users or simultaneous data updates.

The application also does not currently include a login system, role-based permissions, online payment facilities, email notifications, or cloud-based deployment.

The current version is intended to demonstrate the basic working principles of a Flask web application. These limitations have been considered while planning the project, and possible improvements are discussed in the Future Scope chapter.

CHAPTER 4: REQUIREMENT ANALYSIS

4.1 Introduction to Requirement Analysis

Requirement Analysis is an important phase in the software development process. It involves identifying, studying, and documenting the requirements of a system before starting its implementation. The main purpose of requirement analysis is to understand what the system should do, how it should work, and what resources are required for its development.

The CampusConnect – Campus Event Management & Feedback Analyzer project is designed to simplify the management of university events and student participation. Before developing the application, different requirements were identified to ensure that the system could perform its major operations effectively.

The system focuses on event creation, student registration, registration cancellation, student searching, feedback collection, and feedback analysis. It uses Python and Flask for application development, CSV files for storing records, Pandas for analysing feedback, and Matplotlib for graphical representation.

Requirement analysis helps define the functional and non-functional requirements of the application. It also identifies the hardware, software, and data requirements necessary for successful system execution.

4.2 Functional Requirements

Functional requirements describe the operations and services that the system must provide to users. The major functional requirements of the CampusConnect application are described below.

4.2.1 Event Management

The system allows the user to create and manage university events. While creating an event, the user can enter important information such as Event ID, Event Name, Date, Venue, and Maximum Participants.

The system checks whether the Event ID already exists. If a duplicate Event ID is entered, the system prevents the creation of the duplicate event.

4.2.2 Student Registration

The application provides a facility for students to register for available university events. During registration, the system collects student information, including Student ID, Name, Email, and Department, along with the Event ID.

The system verifies the entered Event ID before completing the registration. It also checks whether the student is already registered for the same event and whether the event has reached its maximum participant capacity.

4.2.3 Student Search

The system provides a search facility to find registered student information using the Student ID. After receiving a valid Student ID, the application displays the available student details and the Event IDs associated with that student.

This feature helps users quickly retrieve registration information without manually checking the entire registration file.

4.2.4 Registration Cancellation

The application allows users to cancel an existing student registration. The system identifies the registration using the Student ID and Event ID and removes the matching registration record.

This feature helps maintain updated registration information and makes it easier to manage student participation.

4.2.5 Feedback Collection

After participating in an event, registered students can submit feedback by providing their Student ID, Event ID, and Rating.

The system checks whether the student is registered for the selected event. It also prevents the same student from submitting duplicate feedback for the same event.

4.2.6 Feedback Analysis

The application uses the Pandas library to read and analyse feedback records stored in the CSV file. It calculates the average rating and displays the number of feedback entries.

This feature helps understand student opinions and evaluate the overall response to university events.

4.2.7 Graphical Representation

The system uses Matplotlib to generate a bar chart representing event-wise student participation.

The graphical representation makes it easier to compare participation across different events and understand the registration information visually.

4.3 Non-Functional Requirements

Non-functional requirements describe the quality, usability, and operational characteristics of a software system.

4.3.1 User-Friendly Interface

The application provides a simple web-based interface through which users can access different event management and feedback features. The interface is designed to make the major operations easy to understand and use.

4.3.2 Reliability

The system performs basic validation during event creation, student registration, and feedback submission. It checks duplicate records, invalid Event IDs, event capacity, and registration eligibility before performing the relevant operations.

4.3.3 Data Persistence

The application stores event, student, registration, and feedback information in CSV files. These files allow the application to retain records between normal executions, provided that the files are maintained and not deleted.

4.3.4 Maintainability

The project is divided into different functional sections, such as event management, student registration, feedback collection, and feedback analysis. This structure makes it easier to understand the source code and modify individual features when required.

4.3.5 Performance

The application is designed for a small academic project and a limited amount of campus event data. Its operations, such as searching student records and analysing feedback, are suitable for this project scale.

4.4 Hardware Requirements

The following hardware resources are sufficient for developing and running the CampusConnect application.

Hardware Component	Minimum Requirement
Processor	Intel Core i3 or equivalent
RAM	4 GB
Storage	500 MB free space or more
Display	Standard monitor
Input Devices	Keyboard and Mouse
Network	Not required for basic local execution

4.5 Software Requirements

The following software and technologies are used in the development of the project.

Software / Technology	Purpose
Python	Main programming language
Flask	Web application framework
HTML	Structure of web pages
CSS	Styling and page design
Pandas	Feedback data analysis
Matplotlib	Graph and chart generation
CSV	Data storage
Web Browser	Accessing the application
Code Editor	Writing and editing program code

4.6 Data Requirements

The CampusConnect application uses CSV files to store and manage its information. Each file is responsible for maintaining a particular type of data.

1. events.csv

This file stores event-related information.

- Event ID
- Event Name
- Date
- Venue
- Maximum Participants

2. students.csv

This file stores student information.

- Student ID
- Student Name
- Email
- Department

3. registrations.csv

This file maintains the relationship between students and events.

- Student ID
- Event ID

4. feedback.csv

This file stores feedback submitted by registered students.

- Student ID
- Event ID
- Rating

Using separate CSV files helps organize the information and makes it easier for the application to read, update, and analyse records.

4.7 System Constraints and Limitations

Although the CampusConnect application provides the major features required for campus event management, it has certain limitations.

1. The application uses CSV files instead of a database management system.
2. The system is designed mainly for a small academic project and limited records.
3. The application does not include a user login or role-based authentication module.
4. The system requires the CSV files to be maintained properly for storing and retrieving records.
5. The application is intended for local execution and does not include a verified online deployment facility.

These limitations can be considered for future improvements.

CHAPTER 5: SYSTEM FLOW

5.1 Introduction

System Flow describes the sequence of operations performed by a software application from the beginning to the completion of a particular task. It explains how information moves through different modules and how the system processes user requests.

The CampusConnect – Campus Event Management & Feedback Analyzer follows a structured workflow to manage campus events, student registrations, registration cancellations, and feedback analysis. The system is developed using Python and Flask, with HTML and CSS used for the web interface.

When a user interacts with the application, the request is handled by the Flask application. The system processes the submitted information, performs the required validations, and stores or retrieves data from CSV files. For feedback analysis, Pandas is used to calculate ratings, while Matplotlib generates graphical representations of event participation.

The overall system flow is divided into several stages, which are explained in the following sections.

5.2 Overall System Flow

The overall workflow of CampusConnect begins when the application is started. The user accesses the home page through a web browser and selects the required operation.

The application provides different functions, including:

1. Creating and managing campus events.
2. Registering students for available events.
3. Searching student registration details.
4. Cancelling existing registrations.
5. Collecting feedback from registered students.
6. Analysing feedback using Pandas.
7. Displaying event-wise participation through a Matplotlib chart.

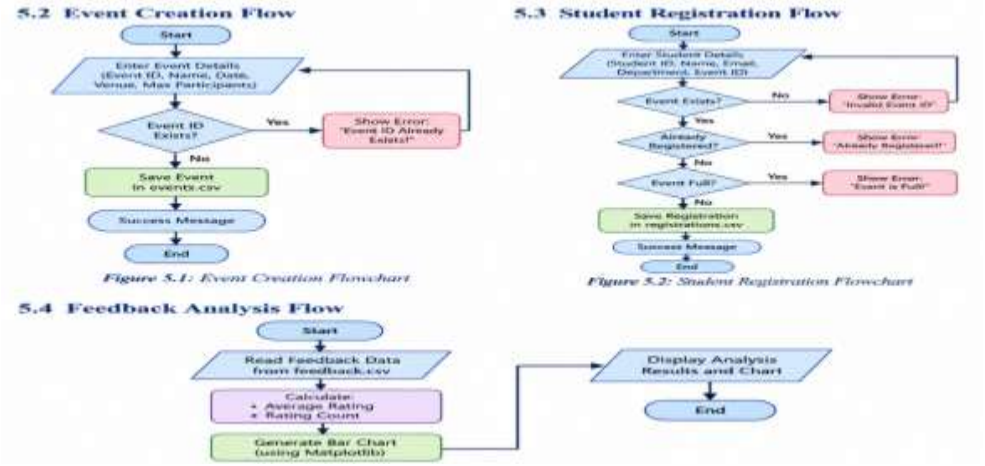


Figure 5.1: Event Creation Flowchart

Figure 5.2: Student Registration Flowchart

5.3 Event Creation Flow

The Event Creation module allows the user to add new campus events to the system. The process starts when the user enters the event details through the web interface.

The system follows these steps:

1. The user opens the event creation section.
2. The user enters the Event ID, Event Name, Date, Venue, and Maximum Participants.
3. The Flask application receives the submitted information.
4. The system checks whether the entered Event ID already exists.
5. If the Event ID is already available, an error message is displayed.
6. If the Event ID is unique, the event details are saved in `events.csv`.
7. A success message is displayed to the user.

This process ensures that duplicate event records are not created.

5.4 Student Registration Flow

The Student Registration module manages the process of registering students for campus events.

The registration process follows these steps:

1. The user enters the Student ID, Student Name, Email, Department, and Event ID.
2. The application checks whether the entered Event ID exists.
3. The system checks whether the student is already registered for the same event.
4. It checks whether the event has reached its maximum participant capacity.
5. If any validation fails, the application displays an appropriate error message.
6. If all conditions are satisfied, the registration information is stored in `registrations.csv`, and the student information is maintained in `students.csv`.
7. A registration success message is displayed.

This process helps prevent invalid registrations, duplicate registrations, and registrations beyond the event capacity.

5.5 Student Search and Registration Cancellation Flow

The system provides facilities to search for student information and cancel existing registrations.

Student Search Flow:

1. The user enters the Student ID.
2. The application searches the student records.
3. If the student is found, the system displays the available student details.

4. The associated Event IDs are also displayed.
5. If no matching student record is found, the system displays an appropriate message.

Registration Cancellation Flow:

1. The user enters the Student ID and Event ID.
2. The application searches for the matching registration.
3. If the registration exists, the system removes the corresponding registration record.
4. If the registration is not found, an appropriate message is displayed.
5. The updated registration information is maintained in the CSV file.

5.6 Feedback Collection Flow

The Feedback Collection module allows registered students to submit their opinions about campus events.

The process includes the following steps:

1. The student enters the Student ID and Event ID.
2. The student provides a rating for the selected event.
3. The system checks whether the student is registered for that event.
4. It checks whether feedback has already been submitted by the same student for the same event.
5. If the student is not registered or duplicate feedback is detected, the system displays an error message.
6. If the validation is successful, the feedback information is saved in `feedback.csv`.
7. A success message confirms that the feedback has been submitted.

This process helps maintain relevant feedback records for further analysis.

5.7 Feedback Analysis Flow

The Feedback Analysis module processes the feedback information stored in the system.

The workflow is described below:

1. The application reads the feedback records from `feedback.csv`.
2. Pandas loads the feedback data into a DataFrame.
3. The system calculates the average feedback rating.
4. It determines the number of feedback entries.
5. The analysis results are displayed through the web interface.
6. Matplotlib is used to generate an event-wise student participation bar chart.
7. The generated chart is displayed on the analysis page.

The analysis helps users understand student feedback and compare participation across different events.

5.8 Data Flow and Storage

The CampusConnect application uses CSV files as its primary data storage mechanism. Different types of information are stored separately to maintain an organized structure.

File Name	Information Stored
`events.csv`	Event details
`students.csv`	Student details
`registrations.csv`	Student and event registration records
`feedback.csv`	Student feedback and ratings

The Flask application handles the communication between the web interface and the CSV files. When a user submits information, the application processes the request and updates the relevant file. When information is requested, the system reads the required records and displays the results.

CHAPTER 6: SYSTEM DESIGN

6.1 Introduction

System Design is an important phase of software development that describes the structure, components, modules, and working mechanism of a software application. It provides a blueprint for transforming the requirements of a project into a functional system.

The CampusConnect – Campus Event Management & Feedback Analyzer is designed as a web-based application using Python and Flask. The system provides different modules for managing campus events, registering students, searching student information, cancelling registrations, collecting feedback, and analysing event-related data.

The system design explains how these modules interact with one another and how information is processed and stored. It also describes the role of the user interface, application logic, data storage, and supporting Python libraries.

6.2 System Architecture

System Architecture represents the overall structure of the application and explains how its major components communicate with one another.

CampusConnect follows a simple web application architecture consisting of three major parts:

6.2.1 Presentation Layer

The Presentation Layer is the user-facing part of the application. It is developed using HTML and CSS.

It provides the web pages and forms through which users can perform different operations, such as:

- Creating new campus events.
- Registering students for events.
- Searching student information.
- Cancelling event registrations.
- Submitting event feedback.
- Viewing feedback analysis and participation charts.

The presentation layer makes the application accessible through a web browser.

6.2.2 Application Layer

The Application Layer is responsible for processing user requests and executing the main operations of the system.

It is developed using Python and Flask. This layer handles the application routes, validates submitted information, performs event and registration operations, and communicates with the CSV files.

It also uses Pandas to analyse feedback records and Matplotlib to generate graphical representations.

6.2.3 Data Storage Layer

The Data Storage Layer maintains the information required by the application.

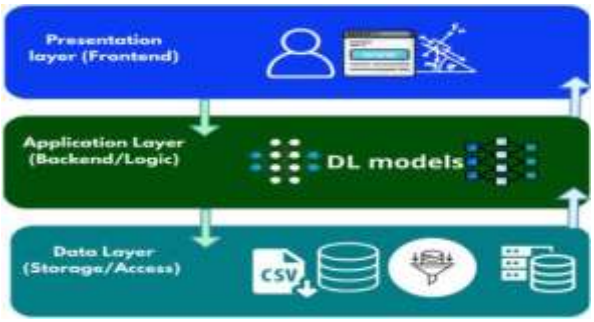
CampusConnect uses CSV files to store different types of information:

- `events.csv` – Stores event details.
- `students.csv` – Stores student information.
- `registrations.csv` – Stores student registration records.
- `feedback.csv` – Stores feedback and rating information.

The application reads information from these files whenever required and updates the relevant files when users perform operations.

6.3 System Architecture Diagram

The following diagram represents the basic architecture of the CampusConnect application.



The diagram shows how the user interacts with the presentation layer, how Flask processes the requests, and how the application communicates with CSV files and analysis libraries.

6.4 Module Design

The CampusConnect application is divided into different modules. Each module performs a specific function and contributes to the overall working of the system.

6.4.1 Event Management Module

The Event Management Module is responsible for creating and maintaining campus event information.

The user can enter details such as Event ID, Event Name, Date, Venue, and Maximum Participants. The system checks the uniqueness of the Event ID before saving the event information.

The event details are stored in `events.csv` for future access.

6.4.2 Student Registration Module

The Student Registration Module allows students to register for available campus events.

The system collects student details and the selected Event ID. Before completing the registration, it checks the validity of the event, duplicate registration, and event capacity.

After successful validation, the student and registration information is maintained in the appropriate CSV files.

6.4.3 Student Search Module

The Student Search Module helps users retrieve student information using the Student ID.

The system searches the stored student records and displays the available student details along with the associated Event IDs.

This module makes it easier to find registration information without manually examining all stored records.

6.4.4 Registration Cancellation Module

The Registration Cancellation Module provides the facility to remove an existing registration.

The user provides the Student ID and Event ID. The application searches for the matching registration and removes it if it exists.

This module helps maintain updated registration records.

6.4.5 Feedback Management Module

The Feedback Management Module allows registered students to submit their ratings for campus events.

The system verifies the student's registration before accepting feedback. It also prevents duplicate feedback submissions for the same student and event.

The feedback information is stored in `feedback.csv`.

6.4.6 Feedback Analysis Module

The Feedback Analysis Module processes the feedback information stored in the system.

The Pandas library is used to read the feedback records and calculate the average rating and total number of feedback entries.

The analysis results help users understand the overall response to different campus events.

6.4.7 Graphical Visualization Module

The Graphical Visualization Module uses Matplotlib to create a bar chart representing event-wise student participation.

The chart provides a visual comparison of participation across different events and makes the information easier to understand.

6.5 Class Design

The project uses object-oriented programming concepts to organize important entities through Python classes.

The main classes used in the application are described below.

6.5.1 Event Class

The Event class represents the details of a campus event.

It contains the following attributes:

- `event_id` – Unique identification of an event.
- `name` – Name of the event.
- `date` – Scheduled date of the event.
- `venue` – Location where the event will take place.
- `max_participants` – Maximum number of students allowed to participate.

This class helps organize and represent event-related information.

6.5.2 Student Class

The Student class represents the information of students participating in campus events.

Its main attributes include:

- `student_id` – Unique identification of a student.
- `name` – Name of the student.
- `email` – Email address of the student.
- `department` – Academic department of the student.

6.5.3 Registration Class

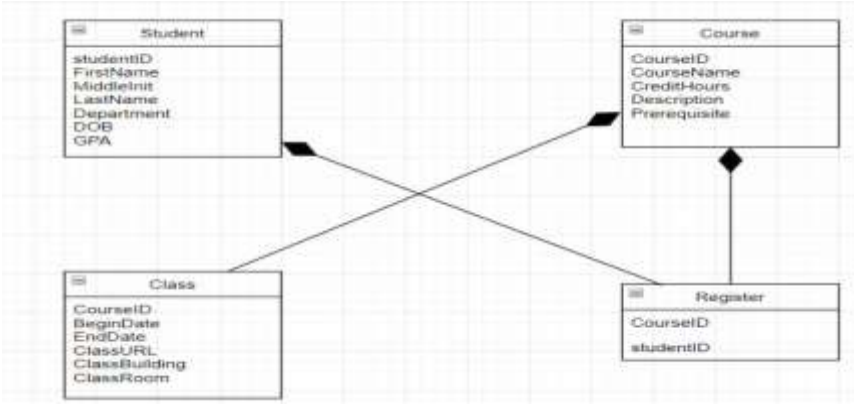
The Registration class represents the relationship between a student and an event.

It contains:

- `student_id` – Identification of the registered student.
- `event_id` – Identification of the selected event.

This class helps represent which student has registered for which event.

6.6 Class Diagram



The Event class represents campus events, while the Student class represents student information. The Registration class connects students with their selected events through Student ID and Event ID.

6.7 Data Design

Data Design explains how information is organized, stored, and maintained within the application.

CampusConnect uses separate CSV files for different categories of information. This approach keeps the data organized and makes it easier for the application to perform reading and writing operations.

CSV File	Main Fields	Purpose
`events.csv`	Event ID, Name, Date, Venue, Maximum Participants	Stores event details
`students.csv`	Student ID, Name, Email, Department	Stores student details
`registrations.csv`	Student ID, Event ID	Maintains registration records
`feedback.csv`	Student ID, Event ID, Rating	Stores feedback records

Separate files help the system maintain different types of information without mixing all records into a single file.

6.8 User Interface Design

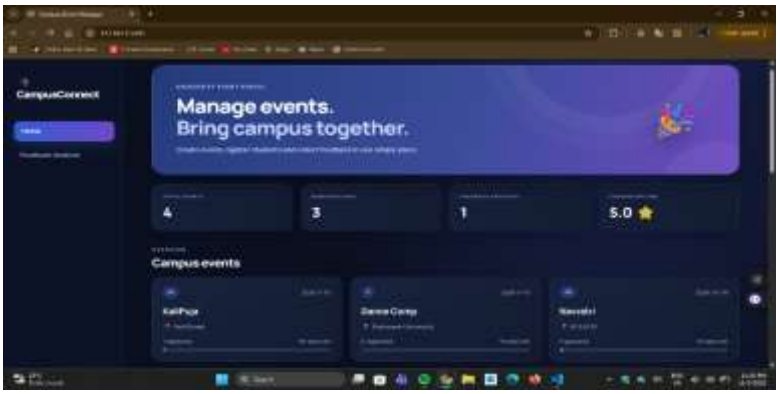
The User Interface is the part of the application through which users interact with the system.

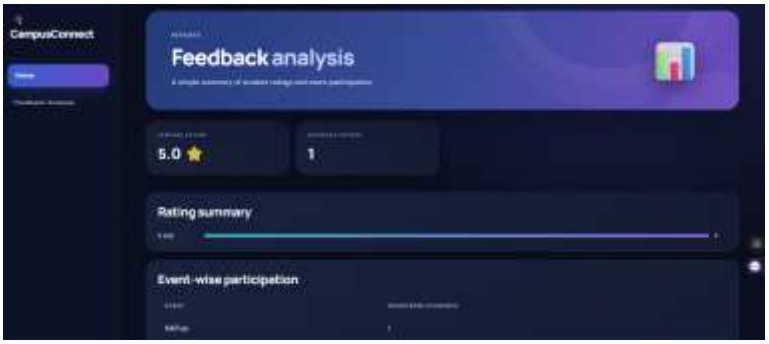
CampusConnect provides a web-based interface developed using HTML and CSS. The interface contains forms and sections for different event management and feedback operations.

The major interface components include:

1. Home Page: Provides access to the main features of the application.
2. Event Management Section: Allows users to create and manage campus events.
3. Student Registration Section: Provides a form for student registration.
4. Student Search Section: Allows users to search for student details.
5. Registration Cancellation Section: Provides options to cancel existing registrations.
6. Feedback Section: Allows registered students to submit ratings.
7. Analysis Page: Displays feedback analysis and graphical information.

For the final report, replace these illustrative designs with screenshots of your own running CampusConnect website.





6.9 Technology Design

The system uses different technologies and libraries to perform its operations.

Technology	Role in System Design
Python	Implements application logic
Flask	Handles web requests and application routes
HTML	Structures web pages and forms
CSS	Provides visual styling
CSV	Stores application records
Pandas	Processes and analyses feedback data
Matplotlib	Generates participation charts

These technologies work together to provide the main functionality of the CampusConnect application.

6.10 Summary

The system design of CampusConnect provides a structured representation of the application's architecture, modules, classes, data storage, and user interface.

The application uses Flask to manage user requests and Python to execute the main operations. CSV files are used to store event, student, registration, and feedback information. Pandas and Matplotlib support feedback analysis and graphical representation.

The modular structure makes the application easier to understand, maintain, and improve. Overall, the system design provides a clear blueprint for the implementation of the CampusConnect application.

CHAPTER 7: EXPERIMENTAL RESULT

7.1 Introduction

Experimental Result is an important part of a software project report. It describes the practical implementation of the developed system and presents the results obtained after executing different modules.

The CampusConnect – Campus Event Management & Feedback Analyzer was developed using Python and Flask. After completing the development process, the application was executed and its major functionalities were checked to observe whether the system performed the expected operations.

The application provides facilities for event creation, student registration, student searching, registration cancellation, feedback collection, and feedback analysis. Different test inputs were provided to verify the working of these features.

The experimental results demonstrate how the application processes user requests, validates information, stores records in CSV files, and displays the required output through the web interface.

7.2 Experimental Environment

The application was developed and tested in a local development environment.

The following technologies and resources were used:

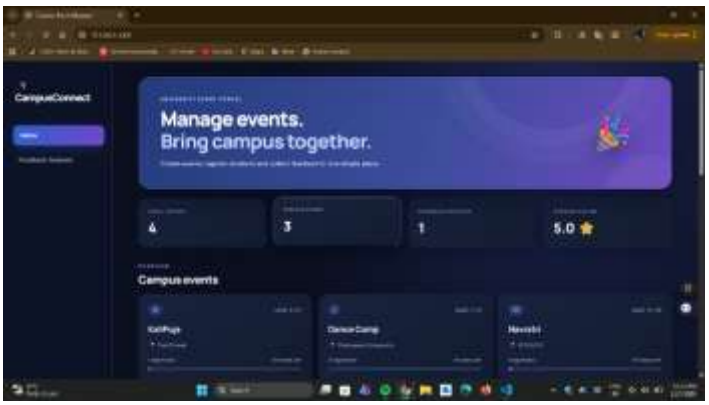
Component	Description
Programming Language	Python
Web Framework	Flask
Frontend	HTML and CSS
Data Storage	CSV Files
Data Analysis	Pandas
Data Visualization	Matplotlib
Execution Environment	Local Computer
Interface	Web Browser

The application was executed locally, and its different modules were tested through the web interface.

7.3 Home Page Result

The home page is the main interface of the CampusConnect application. It provides access to the major functionalities of the system.

When the application starts successfully, the home page is displayed in the web browser. Users can access the event management, student registration, search, cancellation, feedback, and analysis features through the available interface.



7.4 Event Creation Result

The Event Management module was tested by entering sample event information into the event creation form.

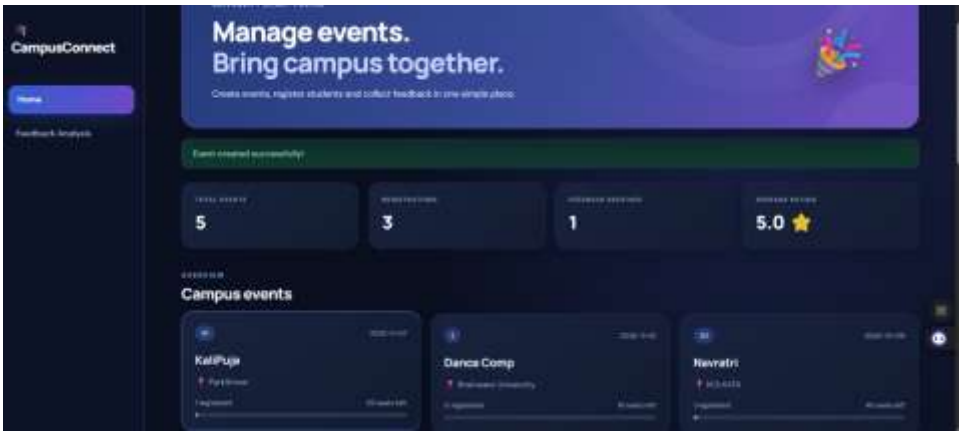
The following sample details were used:

Field	Sample Input
Event ID	E101
Event Name	Dance
Date	20-10-2026
Venue	Main Hall
Maximum Participants	2

After submitting the information, the system checked whether the Event ID already existed. Since the entered Event ID was unique in this test, the event was successfully created and stored in `events.csv`.

Expected successful output

Message: Event created successfully!



Event Creation Output

7.5 Student Registration Result

The Student Registration module was tested by entering student details and selecting an existing event.

Sample student information:

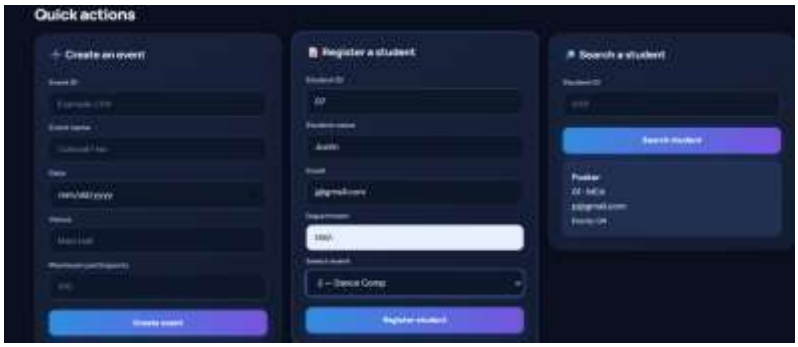
Field	Sample Input
Student ID	S101
Student Name	Puskar
Email	puskar@gmail.com
Department	CSE
Event ID	E101

The system checked the validity of the Event ID, duplicate registration, and available participant capacity before completing the registration.

When all validation conditions were satisfied, the registration was stored successfully.

Expected successful output

Message: Student registered successfully!



Student Registration Result

7.6 Student Search Result

The Student Search module was tested using a valid Student ID.

For example, the Student ID `S102` was entered into the search section. The system searched the stored student records and displayed the matching information.

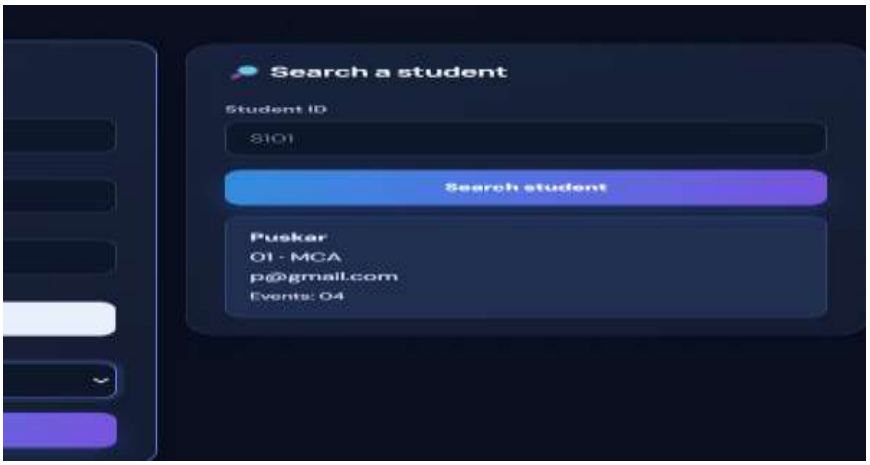
The sample search result was:

Field	Result
Student ID	
Name	Puskar
Email	rahul@gmail.com
Department	MCA
Event ID	01

Search successful

Message: Student Found!

This result demonstrates that the application can retrieve student information and associated event registration details.



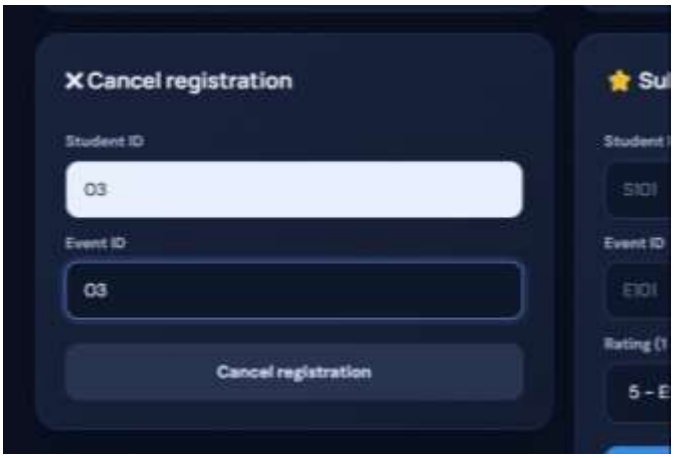
Student Search Result

7.7 Registration Cancellation Result

The Registration Cancellation module was tested by providing the Student ID and Event ID of an existing registration.

The application searched for the matching record in `registrations.csv`. When the registration was found, the system removed the corresponding record and updated the registration information.

This feature helps maintain accurate and updated student participation records.



Registration Cancellation Result

7.8 Feedback Collection Result

The Feedback Collection module was tested by submitting a rating for a registered student.

The following sample information was used:

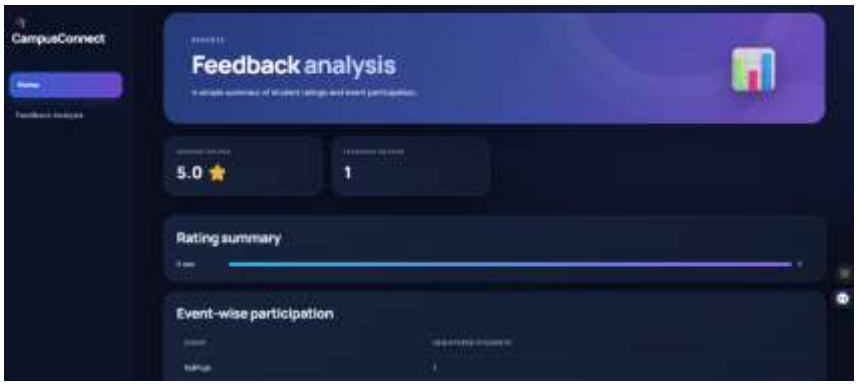
Field	Sample Input
Student ID	S101
Event ID	E101
Rating	5

The system verified the student's registration and checked whether feedback had already been submitted for the selected event.

After successful validation, the feedback record was stored in `feedback.csv`.

Expected successful output

Message: Feedback added successfully!



Feedback Collection Result

7.9 Feedback Analysis Result

The Feedback Analysis module was tested using the feedback information stored in `feedback.csv`.

The Pandas library was used to read the stored feedback records and calculate the average rating and number of feedback entries.

For the sample feedback record:

Student ID	Event ID	Rating
01	E101	5

The analysis produced the following sample results:

Analysis	Result
Average Rating	5.0
Number of Feedback	1

The average rating was calculated from the available feedback records. The result indicates that the sample feedback received a rating of 5.

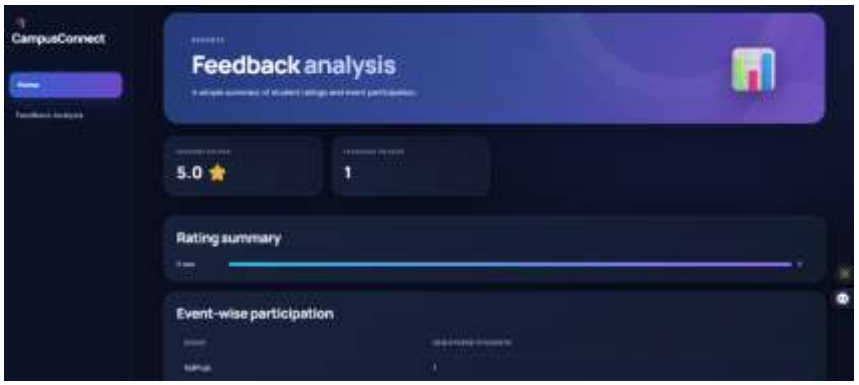


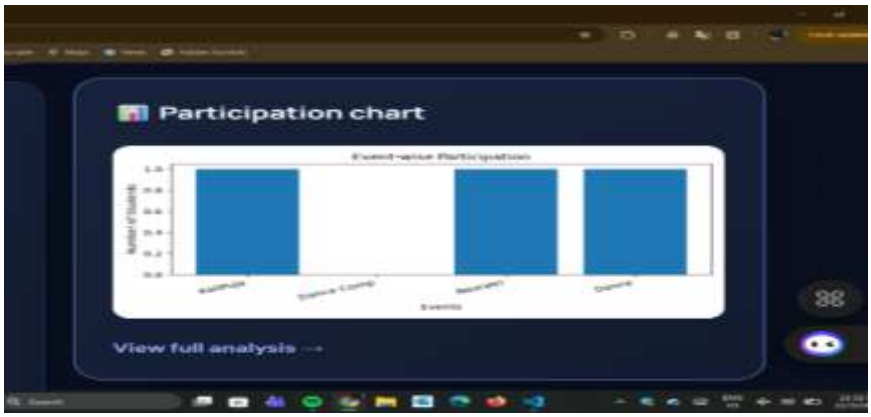
Figure 7.7: Feedback Analysis Result

7.10 Graphical Representation Result

Matplotlib was used to generate a bar chart representing event-wise student participation.

The graph provides a visual comparison of the number of registrations for different events. It helps users understand participation levels more easily than reading individual records.

The generated chart is saved as an image file and displayed on the analysis page.



Event-Wise Participation Chart

7.11 Testing Results

Different test cases were considered to verify the major functionalities of the CampusConnect application.

Test Case	Operation	Expected Behaviour
TC01	Create a new event	Event should be saved
TC02	Enter duplicate Event ID	Duplicate event should be rejected
TC03	Register a student	Valid registration should be saved
TC04	Register the same student again	Duplicate registration should be rejected
TC05	Enter invalid Event ID	Error message should be displayed
TC06	Register when event is full	Registration should be rejected
TC07	Search valid Student ID	Student details should be displayed
TC08	Cancel an existing registration	Registration should be removed
TC09	Submit valid feedback	Feedback should be saved

TC10	Submit duplicate feedback	Duplicate feedback should be rejected
TC11	Analyse feedback	Average rating and feedback count should be displayed
TC12	Generate participation chart	Event-wise bar chart should be generated

These test cases describe the main conditions that should be checked during application testing. The final test status should be filled in after executing each case in the actual application.

7.12 Overall Experimental Outcome

The experimental results show that CampusConnect provides an organized approach to campus event management and feedback analysis.

The event creation module stores event information, while the student registration module maintains student participation records. The search and cancellation modules support registration management.

The feedback module collects ratings from registered students, and the analysis module uses Pandas to calculate important feedback statistics. Matplotlib provides a graphical representation of event-wise participation.

The application demonstrates how Python, Flask, CSV file handling, Pandas, and Matplotlib can be combined to develop a simple web-based campus event management system.

CONCLUSION

The CampusConnect – Campus Event Management & Feedback Analyzer project was developed to provide a simple and organized solution for managing university events and student participation. The main aim of this project was to simplify the process of event creation, student registration, registration management, feedback collection, and feedback analysis through a web-based application.

The application was developed using Python and Flask, with HTML and CSS for the user interface. CSV files were used to store event details, student information, registration records, and feedback data. The system provides important facilities such as creating events, registering students, searching student details, cancelling registrations, and submitting event feedback.

The application also performs validation to prevent duplicate event creation, duplicate student registration, invalid event selection, and registration beyond the maximum participant capacity. These features help maintain accurate and organized records.

Furthermore, the Pandas library is used to analyse feedback data by calculating the average rating and the total number of feedback entries. Matplotlib is used to generate graphical representations of event-wise student participation, making the analysis easier to understand.

During the development of this project, practical knowledge was gained in Python programming, Flask web development, file handling, object-oriented programming, data analysis, and graphical visualization. The project also provided an opportunity to understand the different stages of software development, including planning, requirement analysis, system design, implementation, and testing.

Although the current application is designed for a small academic environment, it provides a foundation for future improvements such as database integration, user authentication, online deployment, email notifications, and advanced feedback analysis.

In conclusion, CampusConnect demonstrates how different programming technologies can be combined to develop a useful campus event management system. It achieves its main academic objectives and provides valuable practical experience in software development and data management.